



BEAST
CODE. THINK. DOMINATE.

BEAST INFERENCE ECONOMY INVERSION

Adaptive Inference Engine and
Lattice-Crystallized Runtime Build

WORKING PAPER



CYBER GOVERNANCE
Secure • Accountable • Resilient



SYSTEMS ENGINEERING
Modular • Composable • Verifiable



ADAPTIVE INFERENCE
Contextual • Efficient • Evolving



ECONOMIC ALIGNMENT
Incentivized • Transparent • Equitable



AUTHOR(S)
TBD

PROJECT
BEAST Framework Initiative

INSTITUTION / AFFILIATION
TBD

DOCUMENT TYPE
Working Paper

VERSION
1.0

DATE
May 2025

“ From inference to inversion,
systems that learn also lead. ”

Abstract

Large language model coding agents increasingly act as autonomous or semi-autonomous software workers, yet their outputs remain difficult to trust, compare, govern, reuse, and cost. BEAST addresses the full lifecycle of agentic inference: input evidence, provider handoff, output Action IR, verification, provider fitness, compute receipts, reusable capability, inference conversion, lattice-crystallized compute, and now an adaptive runtime blueprint for production execution. This v0.8 revision retains the v0.7 evidence build while adding four planned runtime systems: a Unified Adaptive Router, a Local Adapter Compiler, a Defragmentation Daemon, and an experimental Hardware-Bypassed Zero-Copy KV Transport layer. Together these systems bridge Phase 6 crystallization, Phase 7 distillation, Phase 8 adapter/lattice evidence, and the active developer workspace. The runtime design routes exact fingerprint-compatible matches to zero-call crystal proposals, structural matches to proposal-only local specialist adapters, and unresolved cases to provider fallback. The Local Adapter Compiler turns privacy-scrubbed crystal datasets into PEFT/LoRA artifacts and registers low-entropy local specialists only after verifier-gated promotion checks. The Defragmentation Daemon consolidates repetitive trace evidence into high-value lattice nodes while demoting stale crystals under repository or policy drift. The KV transport plan remains experimental and manifest-first, because cache reuse carries engine-compatibility and privacy risks. Claims remain bounded: BEAST demonstrates verified crystal routing, lattice organization, proposal-only adapter governance, and a production-ready implementation plan; it does not yet claim autonomous adapter execution or production-grade KV cache restoration until the corresponding verifier gauntlets pass.

Keywords: agentic AI; coding agents; output governance; compute governance; provider fitness; inference economy; Action IR; verified software actions; LLM serving; KV cache reuse; semantic compute credit; lattice-crystallized compute; adaptive dispatcher; adapter distillation; LoRA; crystal-only routing; semantic defragmentation; capability treasury

v0.9 final extension. The final section adds the missing commons layer discovered in the NEW(1).zip archive: Semantic Compute Pages, Generative Crystals, chain-witnessed lattice integrity, hardware capability cards, portable .beast-space bundles, a Forge node economy, and a Phase 9 KV transport control plane. These layers extend BEAST from a single governed runtime toward a governed compute commons while preserving the same execution boundary: crystals, pages, adapters, and spaces may propose; BEAST verifiers decide.

1. Introduction

AI coding agents are commonly evaluated by whether they can produce patches, call tools, and pass tests. That framing is useful but incomplete. In practical software workflows, the important question is not only whether a model can produce a plausible answer. It is whether a system can govern the full path from task evidence to verified software action, and whether the probabilistic inference used along the way was justified.

The compute-pressure context makes this question urgent. The International Energy Agency projects that global data-centre electricity consumption could reach around 945 TWh by 2030 in its base case, with data-centre electricity demand growing around 15% per year from 2024 to 2030 [4]. At the same time, serving systems are being optimized through provider routing, continuous batching, PagedAttention, prefix caching,

BEAST Inference Economy Inversion Working Paper v0.9 | Verified Compute Commons and Final Runtime Layer and KV-cache reuse [5-9]. These are important infrastructure-level optimizations. BEAST adds a complementary governance question: what semantic uncertainty remains, and does it require inference at all?

This paper argues that agentic coding systems require governance beyond model ranking. A provider may be useful even when it is not independently clean, provided the surrounding system can validate, repair, and verify its output. Conversely, a provider may be fast or cheap while failing output-contract compliance or hidden tests. BEAST therefore evaluates provider roles, output validity, verification behavior, compute receipts, and stored inference value as separate objects.

Inference economy inversion

The current AI economy treats inference as a consumed service call. BEAST treats verified inference residue as a candidate local asset. The economic unit shifts from tokens consumed to verified uncertainty reduced and safely reusable.

2. Contributions

- A governance architecture for agentic coding workflows that spans input evidence, provider handoff, output Action IR, verification, provider fitness, and compute accounting.
- A definition of output governance as the transformation of model output into scoped, typed, verifiable software actions rather than trusted free-form text.
- A provider-fitness framing that separates clean provider completion, BEAST-rescued completion, hidden-clean behavior, route health, latency, tokens, and first-party cost.
- An inference Compute Governor that observes model-call demand, records receipts, applies an ambiguity-escalation policy, and distinguishes counterfactual estimates from realized savings.
- A Phase 1-7 evidence package showing the path from shadow accounting to deterministic displacement, false-reuse detection, adaptive routing, streaming interception, capability crystallization, and durable inference replay.
- A Definitive Mega-Test v0.1 evidence bundle showing within-provider deterministic crystallization through occurrence 5, cross-provider zero-call capability consumption, mutation-triggered stale-reuse blocking, and explicit artifact limitations.
- A theory of inference conversion and crystallized compute, defining how provider calls become outcome evidence, negative capability memory, friction profiles, counterfactual route hypotheses, escrow records, temporal forks, and durable reusable local compute assets.
- A Phase 8 crystal-to-adapter distillation layer that converts verified crystals into a privacy-safe capability lattice, vectorized representations, low-rank adapter-style matrices, a PEFT/LoRA-ready package, and held-out proposal gauntlet evidence under strict proposal-only authority.
- A production runtime blueprint that operationalizes the inversion through a Unified Adaptive Router, Local Adapter Compiler, Defragmentation Daemon, and experimental zero-copy KV transport boundary, while preserving proposal-only authority for crystals and adapters.
- A final verified-compute-commons layer that distinguishes Semantic Compute Pages, Generative Crystals, chain-witnessed lattice ledgers, portable .beast-space bundles, hardware adapter capability cards, Forge-node production, and experimental KV transport from production-ready execution claims.

3. Background and Related Work

3.1 Coding-agent evaluation

SWE-bench and SWE-bench Verified have become major reference points for evaluating software-engineering agents. SWE-bench Verified is described as a human-filtered subset of 500 instances intended to improve clarity, solvability, and test correctness [1]. BEAST does not replace such external benchmarks; rather, it studies the governance machinery around an agentic coding workflow. This is why the paper reports clean versus rescued completion, route failures, and hidden-clean rates rather than only pass/fail outcomes.

3.2 Structured outputs and tool governance

Structured-output systems such as OpenAI Structured Outputs constrain model responses to JSON Schema [2]. The Model Context Protocol standardizes connections between LLM applications and external data sources and tools [3]. BEAST builds on this broader movement toward schema-bound agent operation, but its output-governance layer goes further: a schema-valid object must still resolve to allowed paths, compile into a patch or local action, pass visible and hidden tests, respect rollback and security checks, and leave an evidence trail.

3.3 Serving efficiency and stored compute

Hugging Face Text Generation Inference exposes production serving features such as token streaming, distributed tracing, tensor parallelism, continuous batching, Flash Attention, and PagedAttention [5]. Hugging Face Inference Providers also expose many models through a provider fabric [6], while continuous batching keeps GPUs busy by dynamically rescheduling request batches as earlier requests finish [7]. vLLM Automatic Prefix Caching reuses KV cache blocks when requests share a prefix [8]. LMCache extends this direction by storing and sharing KV caches across engines and queries, framing KV cache as a storage and communication medium for inference engines [9]. BEAST is complementary: it treats stored inference not only as engine-level cache, but as verified semantic work that may become a fingerprint-bound local capability.

The crystallized-compute framing further distinguishes BEAST from text caches and engine-level KV reuse. Prompt and KV-cache systems reduce repeated token computation when a prefix, route, or physical cache state can be reused. BEAST instead asks whether a verified software action, route failure, friction profile, or local transform can become a governed semantic asset. Its reuse criterion is therefore not textual similarity alone; it is behavior preservation under policy, repository fingerprint, verifier, and rollback constraints [E15-E16].

3.4 Cost-aware routing and cascade systems

Cost-aware routing and cascade systems are close neighbors to BEAST. RouteLLM learns router models that select between stronger and weaker LLMs to optimize cost-quality trade-offs [10]. FrugalGPT proposes prompt adaptation, LLM approximation, and LLM cascades to reduce inference cost while preserving or improving task performance [11]. BEAST is adjacent to these systems but differs in its unit of control: RouteLLM and FrugalGPT optimize which model or cascade should answer, whereas BEAST governs the full agentic inference lifecycle, including scoped evidence before inference, typed Action IR after inference, local verification, clean-versus-rescued provider fitness, compute receipts, and fingerprint-bound reuse of verified work.

4. BEAST Architecture

BEAST is an agentic inference-governance layer wrapped around coding agents and provider APIs. Its workflow is: task request -> input governance -> provider handoff -> compute governor -> provider/local route -> output governance -> Action IR resolution -> patch compilation -> verification -> Chronicle/evidence recording -> provider-fitness and compute-ledger update.

Figure 1. BEAST governance stack. BEAST governs evidence before inference, actions after inference, provider role assignment, compute necessity, and fingerprint-bound capability reuse.

Layer	Governance question	Representative artifacts
Input governance	Controls what evidence enters the model.	Task envelopes, scoped files, context packets, policy constraints.
Output governance	Controls what actions leave the model.	Action IR, path guards, schema checks, patch compilation, rollback, tests.
Provider fitness	Controls which provider deserves which role.	Clean/rescued completion, hidden clean, route health, cost/fix, latency.
Compute governance	Controls whether inference is necessary.	Plans, gates, receipts, deterministic/reuse candidates, suppression policy.
Capability treasury	Controls when verified work can be reused.	Impact Fingerprints, proofs, demotion, stored inference credits.

Table 1. BEAST governance stack.

5. Research Questions

Research question	Operational question
RQ1 Output governance	Can model output be converted into scoped, typed, verifiable software actions rather than trusted as free-form text?
RQ2 Provider fitness	Can provider performance be separated into clean completion, BEAST-rescued completion, route failure, hidden-test behavior, cost, latency, and task-class fitness?
RQ3 Compute governance	Can BEAST observe inference demand, record complete compute receipts, and estimate or realize avoidable work without degrading verified behavior?
RQ4 Inference economy inversion	Can repeated verified inference outcomes become reusable capabilities rather than disposable responses?
RQ5 Safety under reuse	Do repository, policy, tool-schema, or test drift make stale reuse unavailable before execution?
RQ6 Provider-independent consumption	Can a capability verified under one provider route be consumed under another provider route without another provider call?

Table 2. Research questions.

6. Method

The evidence program now has six layers. First, a Phase 1-7 package tests compute-governance mechanisms: shadow accounting, deterministic displacement, false-reuse detection, adaptive routing, streaming interception, lifecycle demotion, and bounded replay. Second, the combined provider-fitness benchmark evaluates live provider routes as operational roles rather than conventional model rankings. Third, the master mega-test bundle collates staged controlled observations, provider gauntlets, cross-provider reuse cases, mutation recovery cases, receipts, evidence cards, patches, rollback

snapshots, integrity manifests, and explicit artifact limitations. Fourth, the occurrence-10 mutation-recovery Crystal Compute package tests whether the system blocks stale reuse after material drift while still recovering verified behavior through fresh governed execution [E17]. Fifth, the Phase 8 crystal-to-adapter layer tests whether verified crystals can be organized into a capability lattice, vectorized, compiled into adapter-style low-rank parameters, and compared against baseline local-model, wrapper, LoRA-runtime, crystal-only, and cloud-provider lanes [E19-E21]. Sixth, this v0.8 revision specifies the production runtime bridge: adaptive dispatch, continuous local adapter compilation, autonomous semantic defragmentation, and experimental KV transport, all governed by verifier-first proposal-only authority.

The intended controlled design is 6 task families x 5 provider routes x 5 occurrence points x 3 lanes = 450 lane-level observations. The current master bundle is not the completed 450-row experiment. It contains a canonical staged analysis set of 198 non-IID records, plus a 450-row dry-run plan. The paper therefore reports exact descriptive counts rather than inferential p-values or confidence intervals [E14].

6.1 Core metrics

Metric	Meaning	Why it matters
Verified behavior	Visible and hidden tests or bounded verifier agreement.	Used to decide whether work completed safely.
Clean completion	Provider output passed governance without BEAST rescue.	Separates provider capability from system reliability.
Rescued completion	BEAST repaired, canonicalized, or verified imperfect provider output.	Measures governance value without hiding provider weakness.
Receipt coverage	Provider attempts with compute receipts.	Shows accounting completeness.
QPCCD	Quality-preserving cloud-call displacement: Lane C equals or beats Lane B on verified behavior while using fewer cloud calls.	Primary Compute Governor economy signal.
False suppression	Suppressed work that should not have been suppressed.	Redline metric: must remain zero.
False reuse	Reused capability disagrees with live or baseline behavior.	Redline for verified reuse and capability treasury.
Counterfactual avoided tokens/USD	Estimated avoidable work in shadow or comparable-lane mode.	Not realized dollar savings until enforcement and billing evidence prove behavior preservation.

Table 3. Core metrics.

7. Results

7.1 Phase 1-7 evidence

The Phase 1-7 evidence supports a staged claim. Phase 1 is strongest for live shadow accounting and receipt coverage. Phase 2 demonstrates a proof-backed deterministic displacement under an active Impact Fingerprint. Phase 3 deliberately produces and detects a false reuse condition. Phases 4-7 demonstrate bounded mechanisms for adaptive routing, streaming early stop, capability lifecycle, and durable replay. These phases are not broad production claims; they are evidence that the architecture can measure and govern the relevant failure modes.

Phase	Mechanism	Key evidence	Claim boundary
1	Shadow accounting and calibration	xAI live: 24/24 verified, 25/25 receipts, 100% coverage,	Supports live observation, complete receipts, zero

Phase	Mechanism	Key evidence	Claim boundary
		232,081 observed tokens, 33,017 avoidable-token estimate (14.2%), \$0.052214 counterfactual savings (14.5%). Closure: 120 paired attempts, 100% behavior preservation.	suppression, and counterfactual estimates. Does not prove realized savings. [E1-E4]
2	Deterministic displacement	Calibration: 120 paired attempts, 100% calibrated agreement. Live: one Groq shadow call followed by enforced deterministic displacement; <code>provider_execution_request</code> ed=False.	Supports proof-backed displacement for one matching transform only. [E6-E7]
3	Verified reuse and false-reuse redline	Adversarial Groq run: live call succeeded, reuse returned a valid object, but <code>behavior_preserved_against_live_baseline=False</code> ; <code>observed_false_reuse=True</code> .	Negative evidence used correctly: the system can count false reuse. [E8]
4	Adaptive inference and approval	Bounded Groq harness: local adapter selected without provider call, one approval-required pause persisted, one approved provider route executed with 146 tokens.	Supports local route, approval audit, and bounded route behavior. [E9]
5	Streaming interception	Groq stream cancelled after first schema-valid governed object. <code>stream_tokens_saved=80</code> (35.9% of baseline). Repair path succeeded.	Supports early-stop mechanics in a bounded live stream. [E10]
6	Capability crystallization	Before drift: <code>promoted_count=1</code> . After semantic target change: <code>demoted_count=1</code> , <code>state=shadow_revalidation</code> , <code>reusable=False</code> .	Supports lifecycle persistence, fingerprint checking, and demotion under material drift. [E11]
7	Stored inference and runtime replay	Runtime replay displaced two provider calls. Storage metrics: two active credits, <code>measured_reuse_tokens_saved=140</code> , KV adapter payload round-tripped=True.	Supports bounded cached-answer and prefill-identity replay, not general durable KV reuse across production engines. [E12]

Table 4. Phase 1-7 evidence summary.

Read together, the seven phases form a staged argument for inference governance. Phase 1 establishes that BEAST can observe compute demand, attach complete receipts, and preserve behavior without enforcing suppression. Phase 2 shows that probabilistic calls can be displaced only when proof-backed deterministic execution preserves verified behavior. Phase 3 demonstrates the safety redline by refusing stale or adversarial reuse, while Phases 4 to 7 extend the system from routing and streaming control toward capability crystallization and stored inference value, where verified work becomes a fingerprint-bound candidate for future reuse rather than a disposable response.

7.2 Provider-fitness benchmark and route map

The combined live provider benchmark is the clearest RQ2 evidence because it separates system reliability from provider cleanliness. The summary treats each provider route as part of provider identity, reports 22 provider routes, 212 evaluated live tasks, 212/212 BEAST end-to-end completions, 41 clean provider completions, and 171 BEAST-rescued completions [E13]. This is intentionally not a conventional model leaderboard: the operational question is which provider route BEAST should trust for a role, task class, output contract, latency budget, and cost envelope.

Rank	Route	Recommended role	Tasks	Clean/Rescued	Hidden	Fitness
1	xai	clean candidate; cost incomplete	10	5 / 5	50%	0.6757
2	ovhcloud	candidate patch provider	10	5 / 5	20%	0.6632
3	puter_deepseek	candidate patch; high latency	10	4 / 6	20%	0.6191
4	cohere	candidate patch provider	10	4 / 6	0%	0.6140
5	deepinfra	candidate patch; high latency	10	4 / 6	20%	0.6120
6	huggingface	rescue-backed Action IR	10	3 / 7	0%	0.5830
7	nscale	rescue-backed Action IR	10	3 / 7	0%	0.5810
8	mistral	rescue-backed code route	10	2 / 8	0%	0.5447
9	openrouter	fast rescue-backed Action IR	10	2 / 8	0%	0.5440
10	sambanova	fast rescue-backed Action IR	10	1 / 9	0%	0.5117
11	novita	low-clean rescue route	10	1 / 9	0%	0.5098
12	cloudflare	edge microtask / rescue route	10	1 / 9	0%	0.4826
13	featherless	semantic selector candidate	10	2 / 8	0%	0.4221
14	nvidia_nim	refs-only transform selector	2	0 / 2	0%	0.4000
15	cerebras	fast semantic selector	10	2 / 8	0%	0.3966
16	aion_labs	rate-limited rescue route	10	1 / 9	0%	0.3895
17	gemini	experimental rescue route	10	1 / 9	0%	0.3330
18	groq	scout or microtask only	10	0 / 10	0%	0.2300
19	llm7	scout until hash fixed	10	0 / 10	0%	0.2300
20	replicate	route	10	0 / 10	0%	0.2000

Rank	Route	Recommended role	Tasks	Clean/Rescued	Hidden	Fitness
		degraded; exclude				
21	hyperbolic	billing invalid	10	0 / 10	0%	0.2000
22	fal	auth invalid	10	0 / 10	0%	0.2000

Table 5. Compact provider-fitness route map from the combined live benchmark [E13].

The same benchmark defines routing tiers. Candidate patch-provider lanes include xAI, ovhcloud, puter_deepseek, cohere, and deepinfra; rescue-backed Action IR lanes include Hugging Face, Nscale, Mistral, OpenRouter, SambaNova, Novita, and Cloudflare; scout or semantic-selector roles include Cloudflare, Cerebras, Featherless, Groq, LLM7, and NVIDIA NIM; and billing/auth-invalid routes are excluded from capability ranking. A follow-up all-route hidden-cost run tested 25 provider routes with 10 tasks per route, reporting 250/250 BEAST completions, 9/250 clean provider completions, 241/250 BEAST-rescued completions, and 10/10 hidden coverage per route [E13].

7.3 Definitive Mega-Test v0.1 evidence build

The v0.4 revision adds a master evidence bundle generated on 2026-06-21. The bundle is a canonical collation of definitive mega-test runs and supporting provider gauntlets. It contains 907 archive entries, 104 evidence cards, 66 patch files, 34 rollback snapshots, 68 compute receipt or semantic credit files, 272 evidence-index records, an integrity manifest, and a secret scan with zero matches. The bundle explicitly notes that raw provider response bodies were not persisted by the source runs; surviving metadata includes response length, validation status, token usage, latency, and failure reason [E14].

Evidence component	Observed result	Interpretation
Designed controlled matrix	450 planned lane-level rows	Design target: 6 families x 5 providers x 5 occurrence points x 3 lanes. Not yet fully executed.
Canonical staged analysis	198 records, non-IID	Descriptive evidence only; no p-values or statistical generalization.
First live five-provider pass	90 observations; raw 1/30 completed, BEAST without governor 0/30, full BEAST 30/30 with 24 cloud calls	Shows governance/rescue dominance over direct provider output on these fixtures, but Lane C combines multiple interventions.
Mature NVIDIA run	72 observations; Lane C 24/24 completed, 12 cloud calls, 12 deterministic zero-call reuses, QPCCD 12/24 = 0.50	Supports within-provider deterministic crystallization through occurrence 5.
Mutation recovery	6/6 stale reuses blocked, 6/6 recovered, 0 false reuse	Supports stale-fingerprint blocking and recovery through fresh verification.
Cross-provider primary reuse	12/12 cases from NVIDIA NIM to Mistral/CoHere, zero target-provider execution, zero cloud calls, visible/hidden behavior preserved, 0 incorrect reuse, 18,768 avoided tokens estimated	Supports provider-independent consumption of an already verified local capability under the harness.
Groq scout reuse	6/6 cases, 9,384 avoided tokens estimated, 0 false reuse	Supports scout-style consumption of stored capability, not patch-author fitness.
Artifact completeness	104 cards, 66 patches, 34 rollbacks, 68 receipts/credits; raw provider bodies absent	Strong provenance for derived artifacts; raw-response forensic reproduction remains limited.

Table 6. Definitive Mega-Test v0.1 evidence summary [E14].

Figure 2. Mature NVIDIA crystallization trajectory. Occurrences 1 and 2 use cloud calls while evidence is immature; occurrences 3 and 5 show deterministic zero-call reuse across all six task families.

7.4 Provider anchors inside the master bundle

Provider	Completed	Clean	Rescued	Fitness	JSON	Schema	Recommended role
cohere	9/18	4	5	0.6073	1.0000	0.8889	clean_candidate_cost_incomplete
mistral	6/18	1	5	0.5208	1.0000	0.8889	clean_candidate_cost_incomplete
groq	6/18	0	6	0.2833	0.7778	0.1111	scout_or_infra_probe
nvidia_nim	6/18	0	6	0.2667	0.1111	0.0556	refs_only_transform_selector

Table 7. Provider anchor comparison in the master evidence bundle [E14].

Cohere leads this bounded anchor comparison because it produced four clean completions. Mistral is structurally stronger than NVIDIA NIM on JSON/schema validity but produced fewer clean completions than Cohere. NVIDIA NIM and Groq depended entirely on BEAST rescue in these anchors. These scores are route-role evidence, not universal model rankings.

7.5 Synthesis of Phase 1-7, provider fitness, and mega-test evidence

The v0.3 evidence established the architecture and its staged mechanisms. The v0.4 evidence adds the first serious controlled shape of the inference economy inversion. In the mature NVIDIA slice, BEAST pays for inference during early occurrences, accumulates verified history, and then stops calling the provider at occurrences 3 and 5 while preserving visible and hidden verification. The cross-provider runs then show that the consumed object is not the source provider response itself, but a repository-bound capability that can be used under other provider route identities. The mutation run adds the safety counterpart: BEAST does not merely reuse; it also blocks reuse when fingerprints indicate stale or material drift.

v0.4 claim

In the current evidence, BEAST demonstrates within-provider deterministic crystallization through occurrence 5, provider-independent consumption of verified local capability under Mistral and Cohere route identities, and mutation-triggered stale-reuse blocking with recovery through fresh verification. These are bounded, descriptive harness results, not claims of general production economics.

7.6 Statistical analysis of the mature NVIDIA slice

The staged evidence package reports descriptive counts throughout because the 198-record canonical set is non-IID and combines repeated task/provider contexts across exploratory and confirmatory runs. The mature NVIDIA slice is different in kind: 24 paired observations across six task families at occurrence points 1, 2, 3, and 5, with Lane B and Lane C evaluated on identical tasks under the same provider. This paired structure supports McNemar tests on paired binary outcomes and an exact sign test on cloud-call counts. Results are bounded to this slice and do not generalize to the full 450-observation design.

7.6.1 McNemar test - verified completion

McNemar's test evaluates whether Lane B and Lane C differ in verified completion across the 24 paired observations. Lane C completed every case that Lane B failed and failed no case that Lane B completed.

	Lane B: completes	Lane B: does not complete
Lane C: completes	2	22
Lane C: does not complete	0	0

With $b = 22$ and $c = 0$, the continuity-corrected statistic is $\chi^2 = (|b - c| - 1)^2 / (b + c) = 20.05$. At $df = 1$, this exceeds the $p < 0.05$ critical value of 3.841; the result is $p < 0.001$.

7.6.2 McNemar test - hidden test pass rate

Hidden tests are the primary quality-preservation signal. Lane B passed hidden tests on 2/24 observations (8.3%), while Lane C passed hidden tests on 24/24 observations (100%).

	Lane B: hidden pass	Lane B: hidden fail
Lane C: hidden pass	2	22
Lane C: hidden fail	0	0

The McNemar statistic is again $\chi^2 = 20.05$, $p < 0.001$. This directly supports the Sacred Safety Rule within the mature NVIDIA slice: cloud-call displacement did not degrade hidden-test quality; it improved it.

7.6.3 Sign test - cloud-call reduction

The sign test evaluates whether Lane C used fewer cloud calls than Lane B across paired observations. Of 24 pairs, 12 show Lane C using fewer calls, 0 show Lane C using more, and 12 are tied. Excluding ties gives $n_{\text{effective}} = 12$. The exact one-sided binomial p-value is $P(X \geq 12 \mid n = 12, p = 0.5) = 0.000244$.

7.6.4 Wilson confidence interval - QPCCD rate

QPCCD (quality-preserving cloud-call displacement) is the primary Compute Governor economy signal. Of 24 evaluable pairs, 12 are QPCCD wins: $QPCCD = 12/24 = 0.500$. The 95% Wilson confidence interval is [0.314, 0.686]. The interval is wide because $n = 24$, but its lower bound still indicates that more than one in three governed interactions in this slice eliminated a cloud call while preserving or improving verification quality.

7.6.5 Crystallization occurrence consistency

All six task families reached their first deterministic zero-call reuse at exactly occurrence 3. This consistency across distinct task families suggests that occurrence 3 reflects the evidence-accumulation policy rather than an artifact of one family.

Task family	First zero-call occurrence	Lane C completed	Hidden pass
schema_validation	O3	Yes	Yes
provider_alias_normalization	O3	Yes	Yes
patch_compilation	O3	Yes	Yes
syntax_check	O3	Yes	Yes

route_diagnostics	O3	Yes	Yes
secret_redaction	O3	Yes	Yes

Table 8. Crystallization occurrence and verification outcomes per family in the mature NVIDIA slice. All families crystallized at occurrence 3 with full verified and hidden-test completion.

Statistical scope. The tests above apply exclusively to the 24-observation mature NVIDIA slice. The broader 198-record staged package and the first-live pass are reported as descriptive counts only. McNemar tests require paired binary outcomes and sign tests require paired ordinal outcomes; those conditions hold within the mature NVIDIA slice but not across the full non-IID staged dataset.

7.7 Inference conversion and crystallized compute

The v0.7 conceptual shift is that BEAST does not merely govern provider calls; it converts them. Inference conversion means that each probabilistic call is treated as raw material for durable system knowledge. A call can produce a positive capability, a negative capability, a friction profile, a counterfactual route hypothesis, a compute escrow record, a temporal crystal fork, or a durable intelligence artifact. Crystallized compute is the durable product of that conversion: a locally reusable capability object with provenance, behavioral boundary, impact fingerprint, reuse criteria, rollback semantics, lifecycle controls, and cost accounting.

This is not ordinary caching. A cache returns a prior answer because a prompt, prefix, or embedding looks close enough. A BEAST crystal is consumed only when the new task remains inside a verified boundary. The boundary includes the relevant task class, repository state, file and symbol fingerprints, visible and hidden test evidence, provider or route provenance, and policy version. In this sense, BEAST treats uncertainty as a phase-transition process: provider output enters as probabilistic vapor, but only verified residue condenses into local compute.

Figure 3. Inference conversion and crystallized compute. Provider calls are converted into evidence records, verifier outcomes, capability crystals, and durable treasury assets. Zero-call reuse is permitted only when the fingerprint-bound capability remains inside its verified boundary.

Object	Definition	Why it matters
Inference conversion	Transformation of model-call output into persistent evidence: outcomes, failures, friction, receipts, hypotheses, and artifacts.	Makes inference cumulative rather than disposable.
Negative capability	Memory that a provider/route/task/scope repeatedly failed or required too much rescue.	Failure becomes routing intelligence instead of waste.
Friction profile	Latency, schema failure, rescue burden, rollback risk, malformed output, or orchestration drag attached to a route.	Lets routing optimize for operational cost, not only model quality.
Counterfactual crystal	A rejected route preserved as a measurable hypothesis for later execution and resolution.	Unchosen routes become calibratable evidence.
Compute escrow	Budget reserved before inference and settled/refunded against verified	Turns spend into auditable accounting.

	delivery and actual cost.	
Temporal crystal fork	Stable, candidate, or experimental capability branch with bounded traffic, promotion gates, and rollback.	Allows safe experimentation without corrupting production behavior.
Crystallized compute	Verified local compute asset with provenance, impact fingerprint, replay criteria, rollback semantics, and cost accounting.	Allows zero-call local reuse when the future task remains inside the verified boundary.

Table 11. Inference conversion and crystallized-compute objects.

7.8 Crystal Compute phase package

The Crystal Compute phase package extends the master mega-test artifact with six local, shadow, or advisory evidence phases. The package reports passed=True across six phases and explicitly bounds the claim: Phases 1-6 are integrated as local/shadow/advisory evidence, while live provider displacement remains governed by the live mega observations and receipt package [E15].

Phase	Mechanism	Evidence in package	Claim boundary
1	Token/evidence calibration	18 paired attempts; calibration coverage 1.0; avoidable-token arithmetic error 0.0; provider execution preserved.	Calibrates receipt arithmetic; live calibration remains continuous monitoring.
2	Friction-aware routing	9 paired shadow attempts; friction selection would change 3/9 routes; shadow preserved the current route rate 1.0.	Measures displacement pressure; enforcement disabled.
3	Counterfactual crystals	2 counterfactual crystals created and 2 resolved; rejected routes retained as later-testable hypotheses.	Promotion remains advisory.
4	Compute escrow	USD 0.101 reserved; USD 0.041 actual; USD 0.060 refunded; 2/2 settled with verified delivery.	Local reserve/settle/refund accounting; production budget enforcement policy-gated.
5	Temporal crystal forks	Stable/candidate/experimental channels, 25% candidate share, 5% experimental share, rollback after failure, duplicate merge, split, and retirement operations.	Bounded fork lifecycle; production rollout approval-gated.
6	Durable intelligence	Semantic RAID integrity before/after repair, one repaired reference, fossil replay with valid lineage, value-aware GC retained one artifact.	Local shard repair and deterministic replay; distributed object-store replication future work.

Table 12. Crystal Compute Phase 1-6 evidence package [E15].

Taken together, the Crystal Compute phases make the inference-economy claim sharper. The mega-test already shows that verified repetition can displace later cloud calls under bounded conditions. The Crystal Compute layer asks what is created by the calls that do occur: calibration traces, friction adjustments, counterfactual hypotheses, escrow settlements, temporal forks, and durable replayable intelligence. The claim is no longer only that BEAST can avoid selected future calls. It is that BEAST can convert paid uncertainty into an accumulating local compute substrate, while keeping enforcement gated by proof, policy, and live receipts.

7.9 Occurrence-10 mutation recovery and Crystal Compute package

The new occurrence-10 package is a mutation-recovery run, not a displacement run. It intentionally tests whether BEAST refuses stale reuse after material drift, then recovers verified behavior through fresh governed execution. The package contains 18 controlled observations across one provider (nvidia_nim), six families, one occurrence point (O10), and three lanes: raw, BEAST without Compute Governor, and full BEAST with Compute Governor. The Crystal Compute phase package is integrated and passed all six local/shadow/advisory phases.

Lane	Completed	Hidden passed	Cloud calls	Rescued completed
beast_no_compute_governor	0/6	0/6	6	0/6
full_beast_compute_governor	6/6	6/6	6	6/6
raw	0/6	0/6	6	0/6

Table 9. Occurrence-10 mutation-recovery lane summary. QPCCD is 0/6 because the correct safety behavior after material drift is to block stale reuse and revalidate, not to force zero-call reuse.

Mutation tier	Cases	Incorrect reuse	Zero-call local reuse	Shadow revalidation	Cloud/human escalation	Next action summary
Cosmetic	6	0	6	0	0	reuse_local: 6
Semantic adjacent	6	0	0	6	0	shadow_revalidate: 6
Structural/tool-schema	6	0	0	6	0	shadow_revalidate: 6
Breaking target/test	6	0	0	0	6	cloud_or_human_escalation: 6

Table 10. Mutation ladder behavior at occurrence 10. Cosmetic changes remain locally reusable; semantic-adjacent and structural/tool-schema changes require shadow revalidation; breaking target/test changes require cloud or human escalation. All 24 policy decisions record zero incorrect reuse.

The live mutation-recovery subset contains 6 cases across the six task families. It records reuse_blocked_count=6, recovered_count=6, false_reuse_count=0, and demotion_state=shadow_revalidation for all six live recovery cases. The controlled lane comparison shows that raw and BEAST-without-governor lanes completed 0/6, while full BEAST completed 6/6 with 6/6 hidden passes. This result should be interpreted as a safety result rather than an economy result: BEAST chose fresh verification because the capability boundary had been disturbed.

The occurrence-10 package also fixes a previous weakness in the mutation story: it carries real target file hashes, semantic hashes, symbol hashes, test hashes, tool-schema hashes, and policy-version

evidence inside the fingerprint objects. This makes stale-reuse blocking repository-bound rather than merely proof-shaped. The package still records `compute_governor_receipts_present=False` in its acceptance status, so its strongest live displacement and receipt claims should remain tied to the earlier master mega-test bundle.

7.10 Lattice-crystallized compute and semantic defragmentation

The new conceptual layer is lattice-crystallized compute. BEAST does not claim to rewrite or defragment a base model's learned parameter matrices in the current evidence. Instead, it observes the downstream residue of dense parameter activation - prompts, model outputs, repairs, patches, receipts, verifier results, hidden-test outcomes, fingerprints, and negative capability records - and projects the verified portions into a discrete capability lattice. In this sense, the lattice defragments the inference space rather than the model weights themselves: scattered, noisy, provider-specific traces are consolidated into bounded capability nodes that can be promoted, demoted, split, merged, retired, or routed around.

The safety distinction is central. Dense vectors and binary signatures can nominate candidate reuse, but they do not authorize execution. Authorization remains with BEAST policy, repository fingerprints, Action IR constraints, visible and hidden verifiers, and rollback semantics. A useful shorthand is: vectors find neighborhoods; lattices impose order; crystals grant permission.

Figure 4. Phase 8 lattice-to-adapter proof ladder. Dense parameter outputs are converted into governed traces, consolidated into a capability lattice, vectorized into adapter substrate, and consumed either as proposal-only adapter output or zero-token crystal routing.

Layer	What it changes	Governance boundary
Dense parameter activation	Base or provider model produces probabilistic output.	Not reusable by itself; must pass output governance and verification.
Governed trace	Raw output becomes Action IR, patch, receipt, verifier result, rollback evidence, or failure record.	Reusable only as evidence until clustered and bounded.
Capability lattice node	Repeated verified traces are consolidated by task family, action type, path scope, and fingerprint compatibility.	May nominate reuse, demotion, splitting, or adapter training.
Crystal-only route	A deterministic, verifier-gated lattice path emits a proposal without model generation.	May propose inside its boundary; BEAST verifier remains authority.
Adapter distillation substrate	Lattice examples are vectorized and compiled into low-rank matrices or PEFT/LoRA training data.	Adapter may only propose until a held-out verifier gauntlet passes.

Table 14. Lattice-crystallized compute ladder. The current evidence changes downstream compute organization, not base-model parameter matrices directly.

7.11 Phase 8: crystal-to-adapter distillation and held-out proposal gauntlets

The Phase 8 artifact converts verified crystals into a local adapter-training substrate. The distillation package contains 304 privacy-scrubbed local-training rows, a 16-node capability lattice with 304 signals, a fixed-dimensional vectorization with matrix shape 304 x 512, and rank-16 low-rank adapter-style matrices. The low-rank training receipt reports `final_training_accuracy=0.947368`, but its authority is `proposal_only_until_local_verifiers_pass`. The exported true LoRA package targets Qwen/Qwen2.5-0.5B-

Instruct, contains 304 SFT rows, and explicitly states that a trained LoRA must pass BEAST local verifier gauntlets before adoption [E19].

Two held-out gauntlets are now relevant. The first gauntlet (v1.0) measured four lanes across six held-out tasks. Baseline Qwen 0.5B produced parseable task-family-matched output but failed BEAST schema validity and proposal-authority checks across all six tasks. The BEAST Modelfile wrapper produced schema-valid, hidden-verifier-passing, proposal-only governed objects for 6/6 held-out tasks. The LoRA artifact was correctly recognized as non-executable pending an approved runtime harness. The crystal-only route produced schema-valid, hidden-verifier-passing proposals for 6/6 tasks with zero generated tokens and near-zero latency [E20].

The follow-up gauntlet (v1.1) adds a loaded_lora_runtime lane and a disabled cloud-provider lane. It is best interpreted as a runtime smoke test rather than a clean model comparison. The Ollama lanes failed local socket access with ollama_returncode_1 and are excluded from semantic interpretation. The loaded LoRA runtime did execute, but it used a diagnostic random-initialized tiny GPT-2 base and produced unusable non-JSON output, failing all governance checks. This is negative evidence used correctly: the adapter lane remained proposal-only, failed closed, and was not promoted. The crystal-only route again passed 6/6 with zero generated tokens, schema validity 1.0, hidden verifier pass 1.0, proposal-only authority 1.0, and average latency 0.022 ms [E21].

Lane	Evidence state	Schema valid	Hidden pass	Proposal authority	Tokens	Average latency	Interpretation
baseline_qwen_05b	v1.0 measured	0/6	0/6	0/6	382	7338.69 ms	Task-family matched but not BEAST-governed.
beast_model_file_wrapper	v1.0 measured	6/6	6/6	6/6	408	11318.80 ms	Valid proposal-shaping layer.
trained_beast_lora_adapter	v1.0 artifact only	0/6	0/6	6/6	0	0.099 ms	Artifact recognized but not executable.
crystal_only_route	v1.0 measured	6/6	6/6	6/6	0	0.020 ms	Zero-token deterministic proposal path.
trained_beast_lora_adapter	v1.1 loaded runtime	0/6	0/6	0/6	1072	4540.64 ms	Diagnostic random-base runtime failed closed.
crystal_only_route	v1.1 measured	6/6	6/6	6/6	0	0.022 ms	Zero-token route reproduced.

Table 15. Phase 8 held-out proposal gauntlet summary. The v1.1 Ollama lanes are excluded because local socket access failed; cloud was disabled.

The Phase 8 result therefore does not yet prove promoted weight-level behavior change. It does prove a narrower and important ladder: verified crystals can be consolidated into a lattice and adapter training substrate; a wrapper can shape a small local model into BEAST-valid proposal format; a bad or incompatible loaded adapter fails closed; and crystal-only routing can bypass both cloud and local token generation when the task boundary is already represented in the lattice.

7.12 Production runtime blueprint: adaptive inference engine

The v0.8 runtime blueprint moves BEAST from evidence accumulation toward an active, self-healing, low-entropy production engine. The central design shift is that provider inference is no longer the default first action. Incoming EdgeKIR requests are first evaluated against the capability lattice and impact fingerprints. Exact fingerprint-compatible tasks are routed to zero-call crystal proposals; structurally similar tasks are routed to a proposal-only local specialist adapter; and unresolved tasks fall back to provider routing with provider-fitness cooldown and escalation rules. In every case, execution authority remains outside the model: BEAST verifiers decide.

Figure 6. BEAST Runtime v0.8 adaptive inference engine. The production bridge routes incoming work through crystal proposals, local specialist adapters, provider fallback, and verifier authority rather than treating cloud inference as the default first step.

Runtime system	Gap addressed	Primary output	Claim boundary
Unified Adaptive Router	Binary exact replay or full cloud fallback is too coarse.	Route decision: crystal_replay, local_specialist_adapter, or provider_fallback.	Similarity never grants execution; it only selects proposal route.
Local Adapter Compiler	Distillation packages exist, but must become physical LoRA artifacts and local specialist registrations.	PEFT/LoRA artifact, Ollama/PEFT specialist manifest, promotion report.	Rejects random-init or incompatible bases; adapter may only propose.
Defragmentation Daemon	Evidence pollution fragments trace storage and slows lookup.	Consolidated lattice nodes, demotion reports, semantic RAID GC plan.	Never deletes proof receipts; starts in dry-run mode.
Zero-Copy KV Transport	Prefill latency persists when local engines re-tokenize known prefixes.	KV manifest and experimental mmap/state-restore candidate.	Phase 9 experimental only; privacy and engine identity gates required.

Table 14. Production runtime implementation plan and claim boundaries for BEAST v0.8.

7.12.1 Unified Adaptive Router

The Unified Adaptive Router is the runtime dispatcher that evaluates each incoming EdgeKIR request against the lattice before provider inference is invoked. The corrected routing rule is strict: similarity greater than or equal to 0.95 is eligible for exact fingerprint-compatible crystal replay; similarity between 0.80 and 0.95 is a structural match that may select a local specialist adapter; similarity below 0.80 falls back to normal provider routing. The key correction is that exact replay requires high similarity plus boundary compatibility. Weak similarity must never trigger replay.

Figure 7. Corrected dispatcher thresholds. Exact replay is only possible for high-similarity, fingerprint-compatible crystals; structural matches route to adapter proposals; low similarity falls back to providers.

Similarity band	Runtime interpretation	Route	Safety gate
≥ 0.95 plus exact fingerprint compatibility	Known task inside verified crystal boundary.	Zero-call crystal proposal.	Verifier still decides; stale fingerprints demote.
0.80 to < 0.95	Structural match to a lattice node, but not exact replay.	Local specialist adapter proposal.	Adapter cannot execute; hidden tests and policy checks decide.
< 0.80	Insufficient evidence for local reuse or specialist path.	Provider fallback with provider-fitness routing.	Provider output enters Action IR and full verification.

Table 15. Adaptive routing decision bands for the v0.8 dispatcher.

7.12.2 Local Adapter Compiler

The Local Adapter Compiler is the continuous distillation loop that turns a crystal dataset into a physical proposal model. The compiler consumes a privacy-scrubbed distillation package, validates that sufficient positive and negative crystals exist, trains a PEFT/LoRA adapter against a real base model such as Qwen/Qwen2.5-0.5B-Instruct or Llama-3.2-1B-Instruct, rejects diagnostic random-initialized bases, and emits an adapter manifest. Registration into Ollama or another local engine is a secondary step after PEFT runtime validation. This ordering prevents the system from mistaking a loaded adapter artifact for a meaningful language-runtime result.

Figure 8. Crystallized compute as an asset. The adapter compiler consumes proof-backed crystals, semantic credits, provenance boundaries, and negative capability memory rather than raw prompt text alone.

7.12.3 Defragmentation Daemon

The Defragmentation Daemon is the maintenance loop for evidence entropy. High-velocity agent operation creates repetitive trace logs, evidence cards, rollback files, and patch artifacts. The daemon clusters similar traces, consolidates redundant evidence into lattice nodes, computes a Useful Reduction Score, and demotes stale crystals when repository, policy, tool-schema, or hidden-test drift changes the verified boundary. Physical pruning is deliberately conservative: proof receipts, semantic credits, promotion verdicts, hidden-verifier results, rollback snapshots, and fingerprint ledgers are retained. The first production implementation should run in dry-run mode before any deletion is enabled.

Figure 9. Semantic work queue. Repeated probabilistic work becomes a governed queue of compute notes, cloud inference, Action IR, impact fingerprints, and stored crystal evidence.

7.12.4 Hardware-bypassed zero-copy KV transport

The KV transport layer is the most experimental component. It aims to reduce prefill latency by sharing known prefix state through memory-mapped or engine-native KV state objects. Because KV cache reuse can carry privacy, tokenizer, prompt-identity, and engine-compatibility risks, this paper treats zero-copy KV transport as a Phase 9 experimental branch, not a production dependency for v0.8. The first implementation should be manifest-only: model identity, tokenizer hash, system-prompt hash, prefix hash, engine identity, privacy boundary, and reuse_allowed=false until restore correctness is verified.

Figure 10. Code-to-crystal process. The local capability path hardens inference residue into tested outputs, patches, semantic credits, and reusable capability under explicit boundaries.

Runtime contract	
adapter_can_execute = false adapter_may_only_propose = true crystal_may_only_propose = true beast_verifiers_decide = true no_compute_saving_counts_unless_verification_quality_is_preserved = true	

The production contract is intentionally severe: adapters and crystals are not agents of execution. They are proposal mechanisms. BEAST verifiers, path guards, hidden tests, policy state, and rollback checks remain the execution authority. This keeps the inference economy inversion from becoming unsafe automation by another name.

Figure 11. Repeated success becomes infrastructure. BEAST converts verified work into reusable local structures only after evidence, policy, and fingerprints remain aligned.

7.13 Verified Compute Commons: missing layers from NEW(1).zip

The final missing layer is not another provider route or another cache. The NEW(1).zip archive shows BEAST beginning to become a governed compute commons: verified local work can be packaged, witnessed, reproduced, adopted, demoted, and accounted for as bounded compute spaces. This section is added as a final claim-bounded extension because the archive contains evidence for layers that sit above ordinary crystallization and below public marketplace claims.

The archive contains 248 ZIP entries, 212 extracted files, 172 JSON files, 16 Markdown files, 11 JSONL files, six Python files, two NPZ files, two nested ZIP files, one Modelfile, and one PEM key. Its SHA-256 digest is 6956be2f7422ef273a976824539db7fb2bc78833bc70e0066b6e6b75fb33754c. The path-traversal check passed, but the archive must remain internal until sanitized because it contains a private-key fixture and absolute local paths. [E22]

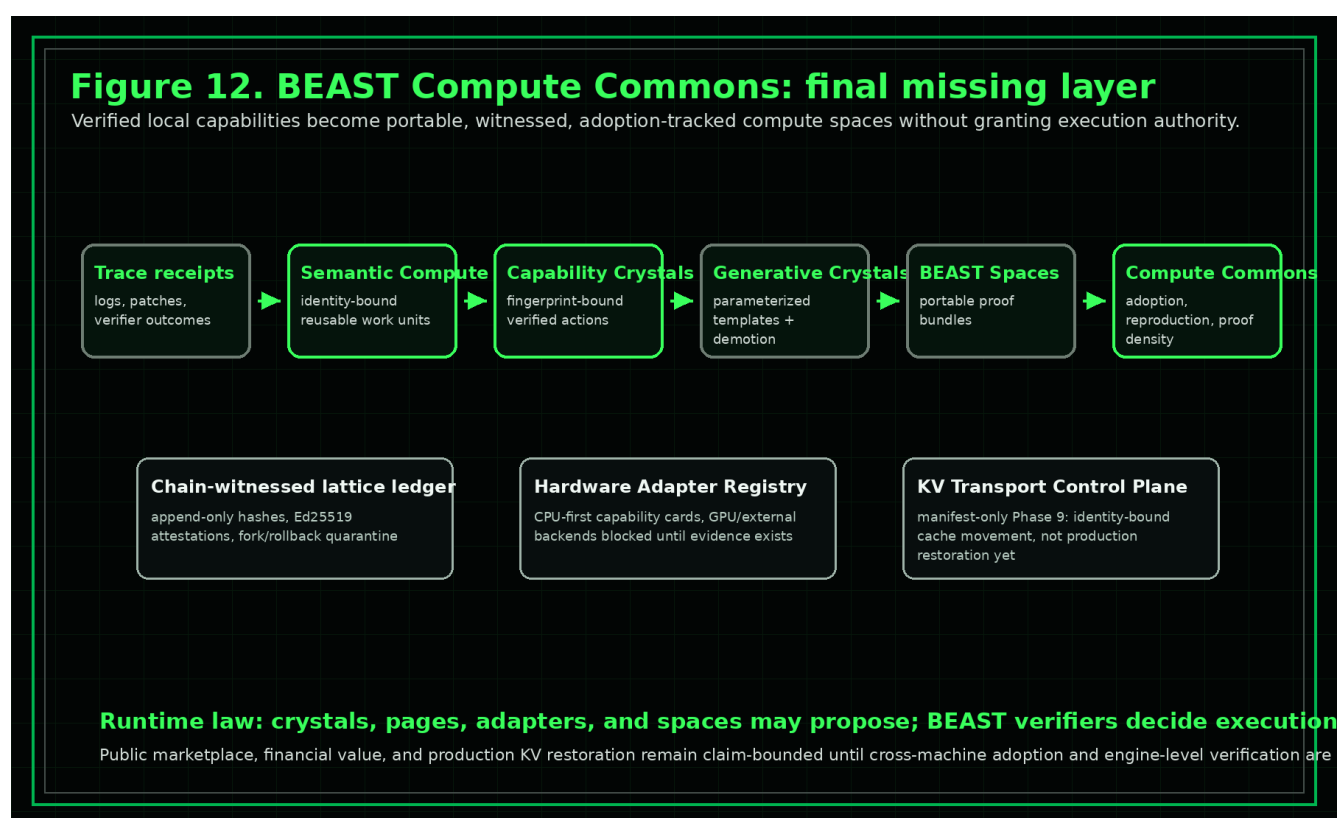


Figure 12. BEAST Compute Commons final missing layer. Verified traces condense into semantic pages, crystals, generative templates, portable BEAST Spaces, and adoption-tracked commons objects, with chain witness, hardware cards, and KV transport remaining under explicit governance rails.

Layer	Observed evidence	What it adds	Claim boundary
BEAST Compute Commons	58 valid spaces; 10 verified spaces; 90 reproductions; 15 adoptions; 10 provider calls avoided; one GPU-avoided space; proof-density threshold closed in the 100-space scenario.	Turns crystals into adoption/reproduction-tracked compute spaces and introduces proof-density economics.	Scenario accounting only; public marketplace, financial value, cross-machine adoption, and production frequency remain unproven.
Semantic Compute Pages	4 active pages; 12 reuse hits across 3 repetitions; 1,458 estimated tokens avoided;	Adds a middle layer between noisy traces and full crystals: small, identity-bound	Page reuse remains identity-bound and verifier-gated; not general text-cache reuse.

	16.953 ms CPU build time; identity mutation gauntlet passed.	reusable semantic work units.	
Generative Crystals	1 active template; 2 successful instantiations; 2 failed receipts; 2 false hits recorded; demotion, rollback, and credit reversal present.	Extends replay into bounded template-instantiation under explicit parameters and demotion.	Only bounded template generalization is supported; no unrestricted autonomous generation.
Chain-witnessed lattice ledger	Ed25519 witness by two nodes; clean audit ok; fork and rollback audits quarantined; defrag does not rewrite history.	Protects lattice integrity against silent rewrite, fork, or rollback.	Archive contains a private-key fixture; public export requires key/path scrubbing.
Hardware Adapter Registry	9 backend cards; CPU-first policy; 1 CPU-ready card; 5 GPU/external cards blocked; Ollama eligible for local probe.	Treats hardware backends as capability cards instead of assumptions.	GPU, vLLM, SGLang, TGI, and external backends are not authoritative until evidence promotes them.
KV Transport Control Plane	786-line skeleton with cache block identity, pin/move/compress/lookup, memory-mapped export/import, and engine-aware metadata.	Begins Phase 9 as a manifest and control API for KV artifacts.	Does not prove production GPU KV restoration or safe cross-engine cache sharing.

Table 14. Missing layers added from the NEW(1).zip archive. The strongest new claim is a governed compute commons; the strongest boundary is that public marketplace, financial value, and production KV transport remain unproven.

7.13.1 Semantic Compute Pages and Generative Crystals

Semantic Compute Pages are the missing middle layer between raw evidence pollution and full capability crystals. They are small, identity-bound fragments of reusable reasoning or orchestration work, consumed only when task identity and mutation checks remain valid. In the archive, four pages validate cleanly, reuse occurs across repeated tasks, and privacy findings are empty. This gives BEAST a way to defragment repeated reasoning without prematurely promoting every repetition into a full crystal. [E22]

Generative Crystals add a second missing capability: bounded generalization. Rather than replaying one exact output, a generative crystal instantiates a verified template across structurally similar tasks. The archive records both successful instantiations and false hits, and it includes demotion, rollback, and credit reversal when the boundary is missed. This is the first evidence that crystals can generalize safely only when their parameters, verifiers, rollback semantics, and demotion logic are explicit. [E22]

7.13.2 Chain-witnessed lattice integrity and hardware capability cards

The chain-witness layer answers a governance question that crystallized compute creates: what prevents the lattice from being silently rewritten after promotion? The archive implements an append-only chain with Ed25519 witness attestations. Clean audit allows promotion, while fork and rollback audits quarantine the ledger. This transforms the lattice from a convenient index into an auditable provenance structure. [E22]

The hardware adapter registry performs the same disciplinary function for runtime backends. A backend is not trusted because it exists in code or appears in a configuration file. It becomes a capability card with state, eligibility, policy, hardware assumptions, and promotion conditions. In the observed registry, Ollama is eligible for local probing, while GPU and external serving engines remain blocked until evidence is produced. [E22]

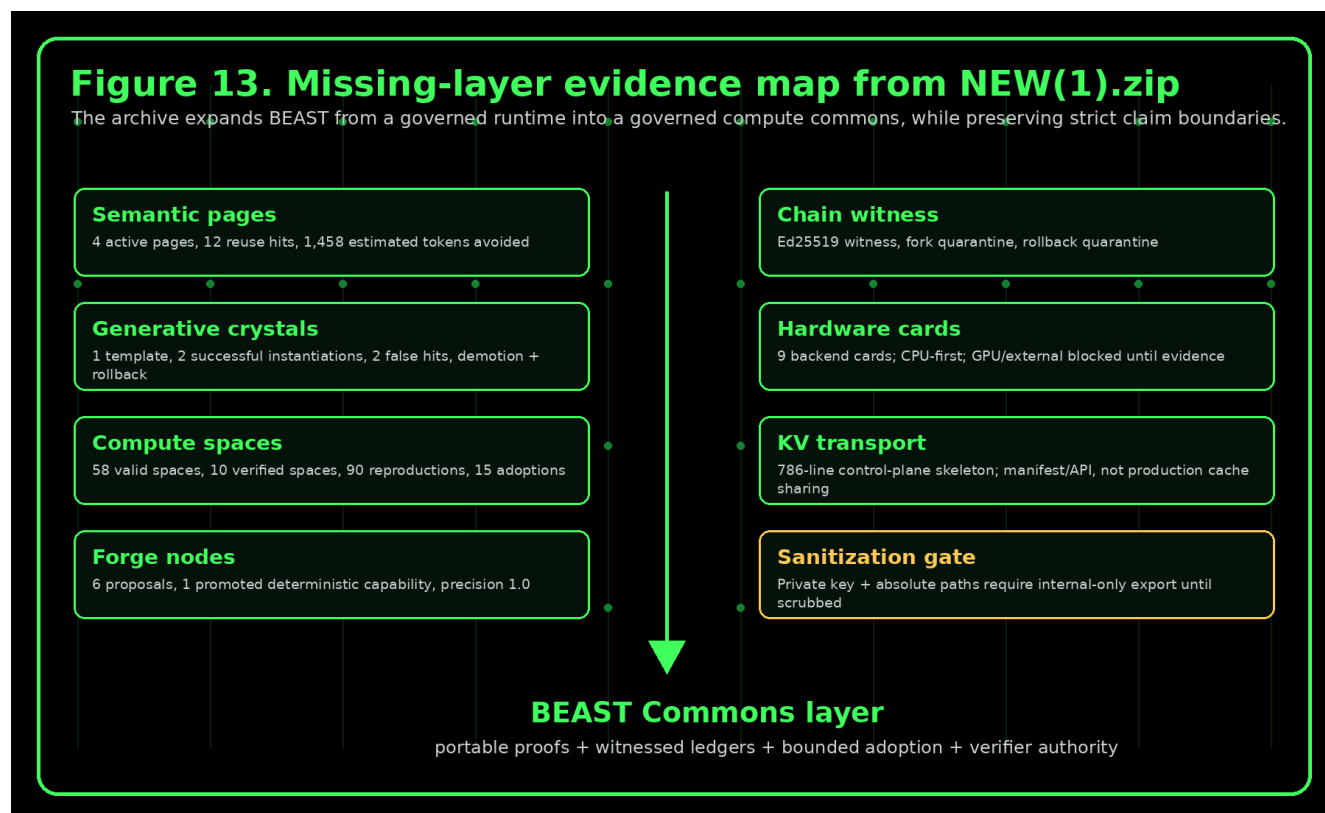


Figure 13. Missing-layer evidence map. The archive adds semantic pages, generative crystals, compute spaces, Forge production, chain witness, hardware capability cards, KV transport, and a sanitization gate.

7.13.3 Portable BEAST Spaces and the Forge node economy

The nested .beast-space bundles are especially important because they define a portable unit for the compute commons. A BEAST Space contains content-addressed artifacts, receipts, normalized plans, reduction evidence, and replay data. The Tiny Llama / Opus gateway repair bundle reports a qwen2.5:0.5b local route, live score 0.955, baseline failure, verification pass, rollback plan, no-cloud-until-local-evidence, secret-redaction gate, and receipt-required policy. This is not merely a result; it is a packaged proof unit. [E22]

The Forge evidence shows how local nodes can produce candidate capabilities without granting them immediate authority. A CPU/Ollama forge node produced six candidate proposals, one deterministic repository-fingerprint capability was promoted, and the local ledger records internal BEAST credits rather than cryptocurrency or redeemable finance. This is the appropriate early economics posture: proof density and reproduction are measured, but monetary value remains scenario-only until adoption across independent machines and production workloads is demonstrated. [E22]

7.13.4 Phase 9 KV transport remains experimental

The archive includes a substantial cross-engine KV cache transport skeleton with cache locations, engine identities, KV cache blocks, pin/move/compress/lookup operations, and memory-mapped import/export APIs. This is stronger than a roadmap paragraph because it defines the objects that a future KV transport layer must govern. However, it is not yet evidence of production GPU KV restoration across heterogeneous engines. The safe claim is that BEAST now has a manifest and control-plane prototype for identity-bound KV artifacts, while production zero-copy KV transport remains Phase 9 experimental work. [E22]

7.13.5 Final synthesis: from runtime to commons

The v0.9 final layer completes the conceptual arc. Phase 6 shows that verified work can crystallize; Phase 7/8 shows that crystals can be organized into lattices and adapter substrates; v0.8 shows how the active runtime should route exact matches, structural matches, and fallback calls; and the NEW(1).zip archive adds the commons layer, where pages, crystals, generative templates, spaces, witnesses, hardware cards, and forge nodes become governed economic objects. The key safety invariant remains unchanged: no compute saving, marketplace listing, adapter route, crystal reuse, or KV transport claim counts unless verification quality remains equal or improves.

v0.9 final claim. BEAST can be framed not only as an inference-governance runtime, but as the early substrate of a governed compute commons: verified work is organized into pages, crystals, templates, spaces, ledgers, and backend cards. These objects may be reproduced, adopted, and valued only under proof-density, witness, mutation, privacy, and verifier constraints. This is a bounded systems claim, not a public marketplace or financial-instrument claim.

8. Discussion: The Inference Economy Inversion

The central idea of BEAST is not only that prompts can be compressed or providers can be routed. Those ideas already exist in modern serving stacks. The deeper inversion is inference conversion: every provider call should either solve the task, teach the system why that route should not be repeated, or create a verified local artifact that can displace future calls. The occurrence-10 mutation package sharpens the safety half of this claim: not every repeated task should reuse a crystal. When fingerprint, policy, tool-schema, or test evidence indicates material drift, the correct economic behavior is to spend again for fresh verification rather than to reuse stale capability. In that framing, the value of inference is not the number of tokens emitted; it is the amount of verified uncertainty reduction that can be preserved without violating safety boundaries.

Conventional frame	BEAST frame
Model calls are rented repeatedly.	Verified work is recorded and may become local capability.
Cost is measured mainly as tokens, latency, and USD.	Cost is measured against verified outcomes, rescue dependency, and future avoidability.
Caching returns a previous answer.	Fingerprint-bound reuse requires behavior, policy, and repository validity.
Serving efficiency asks how to run inference faster.	Compute governance asks whether inference is semantically necessary.
Provider ranking asks which model is best.	Provider fitness asks which provider deserves which role for this task class.

Conventional frame	BEAST frame
Provider transfer means another model produces a similar output.	Provider-independent consumption means a verified local capability is reused without asking the target provider at all.

Table 13. Inference economy inversion.

The crystallized-compute framing adds a sharper balance-sheet analogy: provider failure becomes negative capability memory, unchosen routes become counterfactual crystals, compute spend becomes escrowed and settled, and verified repetition can become a durable local asset rather than another rented token stream.

Paper claim

BEAST reframes probabilistic inference as a governed intervention for unresolved semantic uncertainty. When inference produces verified, fingerprint-bound residue, the result may compound into local capability instead of remaining a disposable service call.

9. Claim Boundaries

The current evidence supports the following claims:

- BEAST can observe live inference demand and record compute receipts with complete coverage in the reported xAI run.
- BEAST can identify counterfactual avoidable tokens and USD in shadow mode while preserving provider execution.
- BEAST can enforce bounded proof-backed deterministic displacement when an active fingerprint and verifier evidence support it.
- BEAST can detect adversarial false reuse when a reused result disagrees with a live baseline.
- BEAST can demonstrate bounded adaptive routing, approval audit, streaming early-stop, capability demotion, and runtime replay mechanisms.
- BEAST can demonstrate within-provider deterministic crystallization through occurrence 5 in the mature NVIDIA slice: Lane C completed 24/24 with 12 cloud calls and 12 deterministic zero-call reuses.
- BEAST can consume a NVIDIA NIM-derived local capability under Mistral and Cohere route identities in 12/12 cross-provider cases without target-provider execution, while preserving visible and hidden behavior.
- BEAST can block stale reuse under live mutation/recovery cases, recover through fresh verification, and record zero false reuse in the reported mutation slice.
- BEAST can locally package inference conversion evidence across six Crystal Compute phases: token calibration, friction-aware routing, counterfactual crystals, compute escrow, temporal forks, and durable intelligence. The current status is local/shadow/advisory unless live displacement is supported by the mega-test receipt package [E15].
- BEAST can demonstrate occurrence-10 mutation safety under a bounded harness: cosmetic drift preserves reuse, semantic-adjacent and structural/tool-schema drift require shadow revalidation, breaking target/test drift requires cloud or human escalation, and all 24 policy decisions record zero incorrect reuse [E17].

BEAST can organize verified inference residue into a privacy-scrubbed capability lattice and vectorization package suitable for adapter training scaffolds [E19].

BEAST can demonstrate a wrapper-shaped local-model proposal source in a held-out six-task gauntlet, where the BEAST Modelfile wrapper passed 6/6 schema and hidden-verifier checks under proposal-only authority [E20].

BEAST can demonstrate a crystal-only route that returns schema-valid, verifier-gated proposals for 6/6 held-out tasks with zero generated tokens and near-zero latency in two gauntlets [E20-E21].

BEAST can fail closed on an incompatible loaded adapter runtime: the v1.1 loaded LoRA diagnostic produced unusable output, failed all governance checks, and was not promoted [E21].

- BEAST can package verified work as portable, content-addressed BEAST Spaces with receipts, plans, reduction evidence, and replay metadata under internal proof-density accounting [E22].
- BEAST can model repeated reasoning as Semantic Compute Pages and bounded Generative Crystals, including false-hit accounting, demotion, rollback, and credit reversal [E22].
- BEAST can protect lattice provenance through append-only chain witness logic and quarantine fork or rollback conditions in the reported local evidence [E22].
- BEAST can represent hardware and runtime backends as promotion-gated capability cards rather than assuming that a backend is authoritative because it is configured [E22].

The current evidence does not yet support the following claims:

Public compute-marketplace viability, financial credit value, cross-machine adoption, production workload frequency, and production KV-cache restoration are not yet proven. The current commons evidence is internal, scenario-based, and must be sanitized before public export [E22].

Promoted Qwen LoRA behavior improvement in live local inference. The true Qwen LoRA training run was initiated, but the completed Qwen-based adapter gauntlet is not yet part of the evidence package.

Direct defragmentation or rearrangement of base-model learned parameter matrices. The present lattice defragments downstream verified inference residue, not the model weights themselves.

Autonomous adapter execution. All adapter and crystal paths remain proposal-only; BEAST verifiers decide execution.

- Completion of the full 450-observation controlled core.
- Natural organic crystallization in uncontrolled no-harness coding sessions.
- Occurrence-10 cloud-call displacement after material drift. The package correctly records QPCCD 0/6 because the safety policy revalidates rather than reuses stale capabilities [E17].
- General realized production savings across arbitrary tasks and providers.
- Broad suppression enforcement in production traffic.
- Universal replacement of cloud inference by deterministic transforms or reuse.
- General safe KV-cache transport across heterogeneous production engines.
- Stable provider superiority independent of routing, billing, rate limits, or model drift.
- Raw-response forensic reproducibility, because raw provider response bodies were not persisted in the source runs.
- Production enforcement of friction routing, counterfactual-crystal promotion, compute escrow budget controls, temporal-fork rollout, or distributed Semantic RAID durability. These remain policy-gated or local/advisory in the Crystal Compute phase package.

10. Threats to Validity and Limitations

The evidence package is strongest as a systems demonstration and staged proof of mechanism. Several limitations remain. First, the full 450-observation controlled core is designed but not yet complete. Second, the canonical master bundle observations are staged and non-IID. Third, natural no-harness

observations are not yet present. Fourth, raw provider response bodies were not persisted in the source runs, limiting forensic reproducibility. Fifth, cost savings remain partly estimated because provider-reported billing fields vary. Sixth, the occurrence-10 mutation package is a safety-recovery test, not a displacement test: QPCCD is expected to be zero because material drift should trigger revalidation or escalation rather than reuse. Seventh, the Crystal Compute phase package is local, shadow, or advisory evidence and does not by itself prove production enforcement or distributed storage durability. These limitations do not weaken the architecture claim; they define the correct empirical frontier for the full ablation matrix. Eighth, Phase 8 separates three claims that must not be collapsed: wrapper-based proposal shaping, crystal-only deterministic routing, and true weight-level adapter improvement. The first two are evidenced in the current held-out gauntlets; the third remains pending until a Qwen-compatible trained LoRA is loaded, measured, and passed through the same verifier gauntlet. Ninth, the v1.1 loaded-adapter smoke test used a diagnostic random-initialized tiny GPT-2 base and is therefore negative runtime-governance evidence, not a valid test of Qwen LoRA quality.

11. Conclusion

BEAST reframes agentic coding inference as a governed lifecycle. It controls what evidence enters the model, what actions can leave it, which providers deserve which roles, whether probabilistic compute is necessary, and whether verified work can become reusable local capability. The Phase 1-7 evidence package shows that this is no longer only a conceptual architecture: BEAST records receipts, calibrates counterfactual estimates, displaces proof-backed live calls in bounded settings, detects false reuse, routes adaptively with approval audit, interrupts streaming after a valid governed object, demotes stale capabilities under drift, and replays bounded durable inference artifacts.

The v0.4 master evidence build pushes the claim further. In one staged controlled package, BEAST shows within-provider deterministic crystallization through occurrence 5, cross-provider zero-call consumption of verified capabilities, and mutation-triggered stale-reuse blocking. The broader contribution is the inference economy inversion. BEAST does not claim that large models or GPU infrastructure are unnecessary. Instead, it asks whether every new task should become a new disposable provider call. It proposes a different accounting: probabilistic compute should be invoked for unresolved uncertainty, verified through behavior, and preserved as local capability only when evidence, policy, and fingerprints remain valid. In a world where inference demand pressures energy, cost, and infrastructure, that question may be as important as building larger models.

The v0.7 contribution is to name and measure the conversion layer. BEAST does not simply cache text or route to cheaper models. It turns repeated, verified inference into an auditable compute substrate: route memories, friction profiles, escrow traces, forked candidate capabilities, and durable crystal objects. The strongest current claim is bounded but unusual: under defined task boundaries, BEAST can locally govern, measure, and convert repeated inference patterns into reusable compute artifacts, while retaining redline controls for stale reuse, false suppression, and repository drift.

The v0.7 addition is therefore not simply more evidence. It clarifies the operating law of crystallized compute: zero-call reuse is valuable only when the capability boundary remains valid; when mutation disturbs the boundary, BEAST must spend compute to regain certainty. This makes the architecture economically interesting without becoming reckless.

The Phase 8 result completes the current proof ladder without overclaiming it. Crystals replace repeated calls when their fingerprints and verifiers authorize reuse. Lattices organize verified residue into reusable capability structure. Adapters may absorb the lattice only as proposal sources until a held-out

verifier gauntlet proves safe behavior. In this framing, BEAST does not merely reduce token consumption; it changes the decision surface of compute by deciding when dense parameter activation, local proposal shaping, crystal-only routing, or external escalation is justified.

The v0.8 runtime blueprint therefore closes the loop between theory and production engineering. Phase 6 crystals preserve verified residue, Phase 7/8 distillation organizes residue into lattices and adapter substrates, and the adaptive runtime uses those assets to decide whether the next task deserves replay, local specialist proposal, provider fallback, or experimental prefill transport. The inversion is no longer only an accounting claim; it becomes an execution policy: spend probabilistic inference only where uncertainty remains, then preserve verified work as local, governed infrastructure.

The final v0.9 layer is therefore the compute-commons layer. BEAST is no longer only a way to route or suppress inference; it is a method for turning verified local work into governed, witnessed, portable, and adoption-tracked compute artifacts. The promise is powerful, but the boundary is equally important: the commons is not a license for autonomous execution, unsanitized sharing, or speculative financialization. It is a disciplined path from repeated inference to reusable capability, from reusable capability to witnessed compute spaces, and from compute spaces to a future economy whose first law remains verification before value.

References

- [1] SWE-bench Verified. Official benchmark description. <https://www.swebench.com/verified.html>
- [2] OpenAI. Structured Outputs. <https://developers.openai.com/api/docs/guides/structured-outputs>
- [3] Model Context Protocol. Specification 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18>
- [4] International Energy Agency. Energy and AI: Energy demand from AI. <https://www.iea.org/reports/energy-and-ai/energy-demand-from-ai>
- [5] Hugging Face. Text Generation Inference documentation. <https://huggingface.co/docs/text-generation-inference/en/index>
- [6] Hugging Face. Inference Providers documentation. <https://huggingface.co/docs/inference-providers/en/index>
- [7] Hugging Face Transformers. Continuous batching. https://huggingface.co/docs/transformers/en/continuous_batching
- [8] vLLM. Automatic Prefix Caching. https://docs.vllm.ai/en/stable/design/prefix_caching/
- [9] Cheng, Y. et al. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665, 2025.
- [10] Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665, 2024.
- [11] Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.
- [12] Bunt, B. EdgeK-BEAST Combined Live Provider Benchmark Summary. GitHub repository artifact, 2026-06-19.
- [13] Bunt, B. Automated Inference Lifecycle Governance and Local Capability Crystallization in Agentic Software Workflows. Internal technical/patent-positioning draft, 2026-06-21.

Evidence Artifacts

- [E1] compute_governor_phase1_xai_live.json. Live xAI shadow observation.
- [E2] compute_governor_phase1_closure.json. Deterministic paired Phase 1 closure benchmark.
- [E3] compute_governor_phase1_calibration.json. Paired usage calibration.
- [E4] compute_governor_phase1_shadow.json. Shadow benchmark.

- [E5] `compute_governor_free_live.json`. Free-provider Phase 1/2 shadow live observation.
- [E6] `compute_governor_phase2_calibration.json`. Phase 2 deterministic paired calibration.
- [E7] `compute_governor_phase2_live_displacement.json`. Groq proof-backed live displacement.
- [E8] `compute_governor_phase3_live_false_reuse.json`. Adversarial false-reuse observation.
- [E9] `compute_governor_phase4_groq_routing.json`. Adaptive routing and approval audit.
- [E10] `compute_governor_phase5_groq_streaming.json`. Streaming interception and repair.
- [E11] `compute_governor_phase6_lifecycle.json`. Capability crystallization and demotion.
- [E12] `compute_governor_phase7_runtime_reuse.json`. Durable runtime replay and KV-prefill identity.
- [E13] `benchmarks/results/live_provider_benchmark_combined_summary.md`. Combined 22-route provider-fitness matrix, follow-up hidden-cost summaries, and runtime routing tiers.
- [E14] `beast_definitive_mega_test_master_evidence.zip`. Master evidence bundle with `README.md`, `ACADEMIC_ASSESSMENT.md`, `analysis_metrics.json`, `coverage_matrix.json`, `provider_anchor_comparison.json`, collated records, receipts, evidence cards, patches, rollback snapshots, and integrity manifest.
- [E15] `crystal_compute_phase_package.json`. Six-phase Crystal Compute package covering token calibration, friction-aware routing, counterfactual crystals, compute escrow, temporal crystal forks, and durable intelligence.
- [E16] `BEAST patent crystal compute.odt`. Technical/patent-positioning draft describing automated inference lifecycle governance and local capability crystallization.
- [E17] `beast_definitive_mega_test_nvidia_mutation_o10_crystal_phase_package.zip`. Occurrence-10 mutation-recovery package with 18 controlled observations, 24 mutation-ladder policy decisions, six live recovery cases, and integrated Crystal Compute Phase 1-6 evidence.

- [E18] `BEAST_Working_Paper_v0.4_final_with_stats-1.pdf`. Prior statistics build used for the stylized front page and mature NVIDIA statistical analysis section.
- [E19] `crystal_to_adapter_distillation.zip`. Phase 8 crystal-to-adapter distillation package with 304 local-training rows, a 16-node capability lattice, vectorized matrix, low-rank adapter-style matrices, and LoRA-ready training scaffold.
- [E20] `Pasted code.json`. Held-out adapter comparison gauntlet v1.0 with baseline Qwen, BEAST Modelfile wrapper, trained-adapter artifact boundary, and crystal-only route.
- [E21] `Pasted code(1).json`. Held-out adapter comparison gauntlet v1.1 with loaded LoRA runtime smoke test, Ollama access failure boundary, cloud-disabled boundary, and crystal-only route validation.
- [E22] `NEW(1).zip`. Deep archive evidence for BEAST Compute Commons, Semantic Compute Pages, Generative Crystals, chain-witnessed lattice ledger, hardware adapter registry, cross-engine KV transport skeleton, Adaptive Dispatcher MVP, Local Adapter Compiler scaffold, portable `.beast-space` bundles, and Forge node economy.